

JAVA PROGRAMMING LABORATORY

Experiment No.: 06

Date: 23/09/26

Title: Demonstration of Hashtable Methods in Java

Objective / Aim

To write and execute a Java program to demonstrate commonly used methods of the Hashtable class, including insertion, retrieval, searching, enumeration of keys and values, finding size, removing entries, and checking whether the Hashtable is empty.

System Requirements

Hardware Requirements

Component	Requirement
Processor	Intel/AMD processor or equivalent
RAM	Minimum 2 GB
Storage	At least 100 MB free space
Keyboard/Monitor	Standard keyboard and display

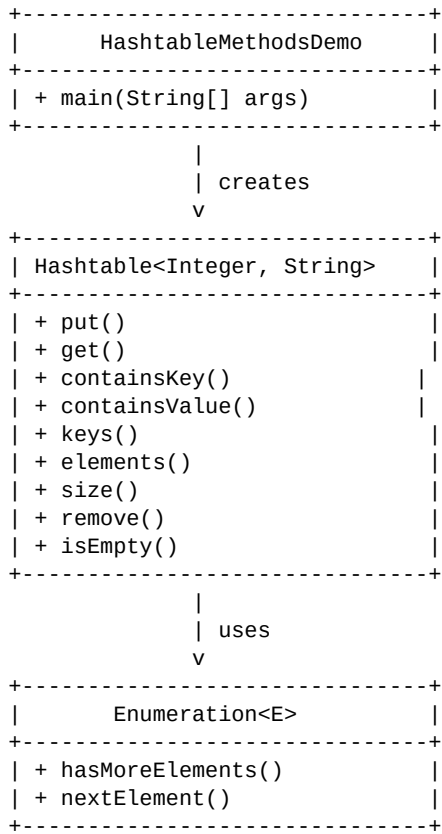
Software Requirements

Software	Requirement
Operating System	Windows / Linux / macOS
Java	JDK 8 or later
Editor	Notepad, VS Code, Eclipse, IntelliJ IDEA, or any Java IDE
Execution	Command Prompt / Terminal or IDE

Algorithm / Logic Description

1. Import Hashtable and Enumeration from java.util.
2. Create a Hashtable with Integer keys and String values.
3. Insert four employee records using put().
4. Display the Hashtable after insertion.
5. Retrieve the employee name for key 102 using get().
6. Check whether key 103 and value "Rahul" exist using containsKey() and containsValue().
7. Obtain all keys using keys() and display them using an Enumeration object.
8. Obtain all values using elements() and display them using an Enumeration object.
9. Find the number of entries using size().
10. Remove the employee with key 103 using remove() and display the updated Hashtable.
11. Check whether the Hashtable is empty using isEmpty().
12. Compile and execute the program and verify the output.

UML Class Diagram



Source Code

```
import java.util.Hashtable;
import java.util Enumeration;

class HashtableMethodsDemo {

    public static void main(String[] args) {

        // Creating a Hashtable
        Hashtable<Integer, String> employees = new Hashtable<>();

        // put(K key, V value)
        employees.put(101, "Rahul");
        employees.put(102, "Priya");
        employees.put(103, "Arjun");
        employees.put(104, "Sneha");

        System.out.println("After put(): " + employees);

        // get(Object key)
        System.out.println("Employee with ID 102: "
            + employees.get(102));

        // containsKey(Object key)
        System.out.println("Contains key 103? "
            + employees.containsKey(103));

        // containsValue(Object value)
        System.out.println("Contains value Rahul? "
            + employees.containsValue("Rahul"));

        // keys()
        System.out.println("Keys:");
        Enumeration<Integer> keyList = employees.keys();

        while (keyList.hasMoreElements()) {
            System.out.println(keyList.nextElement());
        }

        // elements()
        System.out.println("Values:");
        Enumeration<String> valueList = employees.elements();

        while (valueList.hasMoreElements()) {
            System.out.println(valueList.nextElement());
        }

        // size()
        System.out.println("Size of Hashtable: "
            + employees.size());

        // remove(Object key)
        employees.remove(103);
        System.out.println("After remove(103): "
            + employees);

        // isEmpty()
        System.out.println("Is Hashtable empty? "
            + employees.isEmpty());
    }
}
```

Completion and Execution Commands

1. Save the program with the filename **HashtableMethodsDemo.java**.
2. Open Command Prompt or Terminal in the folder containing the file.
3. Compile the program:
`javac HashtableMethodsDemo.java`

4. Execute the program:

```
java HashtableMethodsDemo
```

Output

Note: Hashtable does not guarantee a particular iteration order. Therefore, the order of the displayed entries, keys, and values may differ between executions. The following is one valid output.

```
After put(): {104=Sneha, 103=Arjun, 102=Priya, 101=Rahul}
Employee with ID 102: Priya
Contains key 103? true
Contains value Rahul? true
Keys:
104
103
102
101
Values:
Sneha
Arjun
Priya
Rahul
Size of Hashtable: 4
After remove(103): {104=Sneha, 102=Priya, 101=Rahul}
Is Hashtable empty? false
```

Result / Completion

The Java program was successfully compiled and executed. The commonly used Hashtable methods were demonstrated, including key-value insertion, retrieval, searching, enumeration, size calculation, removal, and empty-state checking.